SER 517 Final Deliverable

Industry Team 9

Personalized Legal Chatbot

Full-stack, containerized legal Q&A and tutoring platform using retrieval-augmented generation, FSPR personalization, document analysis, and privacy-preserving AI processing.

Group Number	Industry Team 9
Group Members	Ayush Desai, Himanshu Pant, Jean Johnson, Mrunal Kapure, Mustakim Shikalgar, Thrupthi H. Manjunatha
Project Name	Personalized Legal Chatbot
Sponsor Name	Dr. John Leddo - MyEdMaster LLC
Repository	https://github.com/Mrunal2148/ser517-industry-9-personalized-legal-chatbot
Video Presentation	https://www.youtube.com/watch?v=7MvoXKHh45g
Submission Date	04/29/2026

Table of Contents

1. Project Description
2. User Manual
3. Installation Guide
4. Issues
5. Outlook
6. Video Presentation

1. Project Description

The Personalized Legal Chatbot is an AI-powered legal education and decision-support product. It helps users understand legal rights, review contracts, ask legal questions, and learn legal concepts through structured tutoring. The system is intended for users who may not have legal training, while this report describes the product for software engineers who need to understand the scope, architecture, runtime behavior, and implementation choices.

The motivation for the project is that legal information is difficult to interpret without context. Generic legal answers can be too broad, too advanced, not jurisdiction-aware, or disconnected

from a user's documents. This product addresses that limitation by combining retrieval-augmented generation, document extraction, user profiling, self-assessment, topic mastery tracking, and conversation summarization. A response is not based only on the current prompt; it is also tailored to the user's jurisdiction, profession, knowledge level, uploaded PDFs, conversation history, and FSPR knowledge gaps.

The application should be understood as an educational and support tool, not as a substitute for a licensed attorney. The system is designed to explain concepts, identify risks, help users ask better questions, and surface issues that a non-expert might miss. Its gotcha questioning behavior is especially important because it proactively highlights traps, hidden obligations, and points that another party may not want the user to notice.

The system is implemented as three application services on a shared Docker network. The React frontend provides the browser interface on port 5173. The Django backend provides the REST API on port 8000 and owns authentication, document upload, preprocessing, persistence, chat management, course progress, and AI service proxying. The FastAPI AI service runs on port 8001 and owns the LangGraph chat pipeline, tutoring pipelines, self-assessment generation and evaluation, FSPR inference, RAG retrieval, PDF extraction, anonymization, and LLM calls. MongoDB stores user and application state, AWS S3 stores uploaded PDFs, Qdrant Cloud stores the legal vector corpus, OpenAI provides GPT-4o and GPT-4o-mini model calls, and LangSmith provides optional observability.

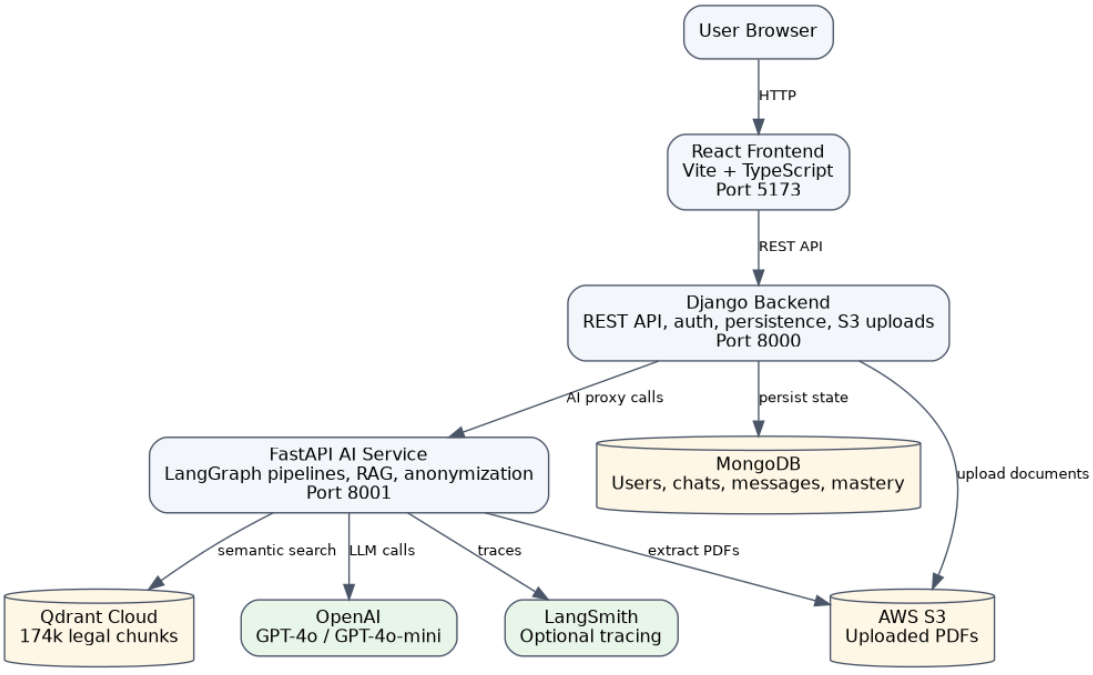


Figure 1. System architecture and request flow. The frontend calls Django, and Django acts as the authenticated application boundary before delegating AI work to FastAPI.

Figure 1 shows the main design decision: the frontend never calls the AI service directly. Django receives all user traffic, authenticates requests, stores messages, manages uploaded

documents, and decides what context should be forwarded. This design keeps application state and security concerns outside the AI service while allowing the AI service to focus on orchestration and model behavior.

Layer	Technology	Purpose
Frontend	React 18, TypeScript, Vite	User interface for authentication, chat, self-assessment, PDFs, and tutoring.
Backend	Django 6.0, Django REST Framework	Application API, persistence, preprocessing, S3 upload, course progress, and AI proxy.
AI Service	FastAPI, LangGraph, LangChain	RAG, extraction, anonymization, reasoning, tutoring, summarization, and FSPR inference.
Data	MongoDB, AWS S3, Qdrant Cloud	Application state, uploaded PDF storage, and legal vector retrieval.
AI/ML	GPT-4o, GPT-4o-mini, legal embedding model	Reasoning, summarization, tutoring, FSPR updates, and semantic search.
Operations	Docker Compose, LangSmith	Local orchestration and optional LLM pipeline tracing.

2. User Manual

Feature	Description	How to use	Sponsor Requirement
User authentication and profile management	Users can register, log in, and maintain a profile. Profile fields such as jurisdiction, profession, and knowledge level provide personalization inputs to the AI service.	Open the frontend, create an account or log in, and complete profile fields before starting legal Q&A.	Yes
Persistent chat history	Users can create, rename, revisit, and delete chat sessions. The backend stores	Use the chat list to create a chat or reopen an existing chat, then continue	Yes

Feature	Description	How to use	Sponsor Requirement
	conversations and messages in MongoDB so users can continue a prior legal discussion.	the conversation.	
Personalized legal Q&A	The main chat feature answers legal questions using the user profile, jurisdiction, profession, FSPR assessment results, topic mastery, uploaded documents, RAG context, and rolling summary.	Ask a legal question in the chat input. Include facts and jurisdiction if they are not already available in the profile.	Yes
FSPR self-assessment	When a new chat begins, the user can complete a four-dimension self-assessment covering Facts, Strategies, Procedures, and Rationales. The system evaluates knowledge gaps and generates a priority profile.	Create a new chat and complete the self-assessment before sending legal questions. The chat also supports fallback behavior if assessment is skipped.	Yes
Self-assessment-driven personalization	After assessment, the AI injects the user's knowledge gaps and priority focus into every later response for that chat. This produces explanations at the right depth rather than generic answers.	Complete the FSPR assessment, then ask a question and review how the response explains weaker dimensions or emphasizes priority gaps.	Yes
Multi-document PDF support	Users can upload one or more PDFs with a query. The AI service	Attach PDFs to a chat message, submit the question, and then	Yes

Feature	Description	How to use	Sponsor Requirement
	extracts text from all relevant PDFs and reasons over them together. Previously uploaded documents can be reused for follow-up questions in the same chat.	ask follow-up questions that refer to the same documents.	
RAG-powered retrieval	The RAG node embeds the user query with a legal-domain embedding model and searches approximately 174k legal chunks from contracts and case law in Qdrant Cloud.	Ask a question that needs legal grounding. The pipeline retrieves the top legal chunks and injects them into the reasoning prompt.	Yes
PII anonymization	The anonymization node uses Microsoft Presidio and spaCy to remove personal identifiers from user text, document text, and retrieved context before data reaches the LLM. Jurisdiction and dollar amounts are preserved because they are legally relevant.	Use the system normally. The privacy step is automatic and occurs before the reasoning node.	Yes
Gotcha questioning	The assistant proactively surfaces risks, traps, hidden obligations, and issues the other party may not want the user to notice.	Ask a contract or legal rights question and review the risk/gotcha portion of the answer.	Yes
Async rolling summarization	Long conversations are compressed into a structured summary once unsummarized	Continue a long chat. The backend triggers summarization in the background and	Yes

Feature	Description	How to use	Sponsor Requirement
	tokens exceed 1500 and more than six messages have accumulated. This preserves context without growing the prompt indefinitely.	future turns use the saved summary.	
Topic mastery tracking	The system tracks legal topic mastery per user and updates scores after interactions. These scores help personalize future chat and learning behavior.	Use chat and course features over time; mastery data is updated by backend and AI-service workflows.	Yes
Structured legal tutoring	The tutoring service supports 20 legal domains at beginner, intermediate, and advanced levels. It generates lessons, MCQ assessments, and personalized feedback. A 70 percent threshold controls level advancement.	Open the courses area, enroll in a domain, generate lesson content, complete lessons, generate an assessment, submit answers, and review feedback.	Yes
FSPR knowledge tracking	Each exchange can update the user's Facts, Strategies, Procedures, and Rationales profile. This gives the product a more granular understanding of user learning than a single score.	Use the chat normally. The AI service performs FSPR inference after exchanges and updates the stored profile.	Yes
LangSmith observability	Developers can enable LangSmith tracing to monitor the	Set LANGCHAIN_TRACING_V2=true and	No / engineering support

Feature	Description	How to use	Sponsor Requirement
	pipeline and inspect LLM calls.	LANGCHAIN_API_KEY in ai/.env, then run the application and inspect traces.	
Containerized execution	The full stack runs through Docker Compose, including frontend, backend, AI service, and MongoDB. No local Python, Node.js, or MongoDB installation is needed.	Configure environment files, run docker compose --profile app up --build, then open http://localhost:5173.	Yes

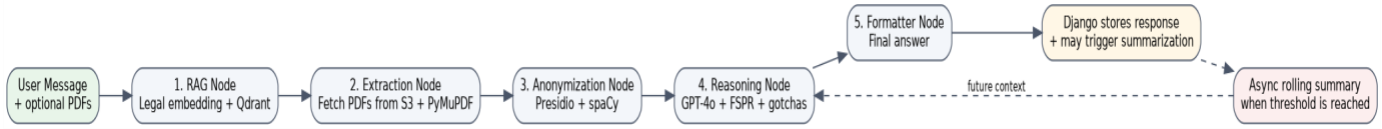


Figure 2. Main chat pipeline. Retrieval, extraction, anonymization, reasoning, and formatting are executed in sequence, while summarization and personalization support long-term context.

In a typical chat flow, the user sends a message through the React interface. Django authenticates the user, stores the message, uploads any PDFs to S3, retrieves relevant conversation context, and calls the AI service. The AI service searches Qdrant, fetches and extracts PDFs, anonymizes text, performs GPT-4o reasoning with personalization inputs, formats the answer, and returns it to Django. Django stores the response and checks whether rolling summarization should be triggered.

The tutoring flow uses separate LangGraph pipelines. Django manages enrollment, progress, caching, and assessment scoring. The AI service generates lesson content, assessment questions, and feedback. This separation allows the same personalization concepts to support both reactive Q&A and structured learning.

3. Installation Guide

This guide explains how to install and run the project from scratch on a fresh VM or computer with only the operating system and Docker Desktop installed. Docker Desktop v4.0 or later is required. The evaluator also needs valid credentials for OpenAI, AWS S3, and Qdrant Cloud. LangSmith is optional.

3.1 Prerequisites

The project is designed so the user does not need to install Python, Node.js, MongoDB, or any local AI runtime. The following external prerequisites are still required: Docker Desktop v4.0 or later, an OpenAI API key, an AWS S3 bucket with credentials, a Qdrant Cloud account and collection, and optional LangSmith credentials for observability.

3.2 Setup Steps

1. Install Docker Desktop v4.0 or later and verify Docker Compose v2 is available.
2. Clone the repository with `git clone https://github.com/Mrunal2148/ser517-industry-9-personalized-legal-chatbot.git` and then run `cd ser517-industry-9-personalized-legal-chatbot`.
3. Create environment files by copying `backend/.env.example` to `backend/.env` and `ai/.env.example` to `ai/.env`. Verify that `client/.env` exists.
4. Configure `backend/.env` with MongoDB, AI service, and S3 values. The AI service URL should be `http://ai:8001` when running inside Docker.
5. Configure `ai/.env` with OpenAI, embedding model, Qdrant, S3, MongoDB, and optional LangSmith values.
6. Configure `client/.env` with `VITE_DJANGO_PROXY_TARGET=http://backend:8000` so the Vite frontend forwards API calls to the Django container.
7. Start all services by running `docker compose --profile app up --build`. The first run builds images and downloads the approximately 737MB legal embedding model into the `ai_model_cache` Docker volume.
8. Open `http://localhost:5173` in a browser after startup completes. The backend is available at `http://localhost:8000` and the AI service is available at `http://localhost:8001` for developer testing.
9. Seed the course catalog in a second terminal by running `docker compose exec backend python manage.py seed_courses`. This creates courses for all 20 domains and 3 levels.
10. Stop or reset the stack as needed. Use `docker compose --profile app down` to stop while preserving data. Use `docker compose --profile app down -v` for a full reset that deletes MongoDB data and the cached model.

3.3 Required Environment Variables

File	Variable	Purpose
backend/.env	MONGODB_URI, MONGODB_HOST, MONGODB_DB_NAME	MongoDB connection and database configuration.
backend/.env	AI_SERVICE_URL	Internal FastAPI service URL, typically http://ai:8001.
backend/.env	AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_STORAGE_BUCKET_NAME, AWS_S3_REGION_NAME	Credentials and configuration for document storage in AWS S3.

File	Variable	Purpose
ai/.env	OPENAI_API_KEY, OPENAI_MODEL	OpenAI access and model selection, with GPT-4o used for main reasoning.
ai/.env	LEGAL_EMBEDDING_MODEL	Legal-domain embedding model, Stern5497/sbert-legal-xlm-roberta-base.
ai/.env	QDRANT_URL, QDRANT_API_KEY, QDRANT_COLLECTION	Qdrant Cloud vector database configuration.
ai/.env	S3_BUCKET_NAME and AWS credentials	S3 access for PDF extraction and presigned URL retrieval.
ai/.env	MONGODB_URI, MONGODB_DB_NAME	MongoDB configuration used by AI-service workflows.
ai/.env	LANGCHAIN_API_KEY, LANGCHAIN_TRACING_V2	Optional LangSmith tracing configuration.
client/.env	VITE_DJANGO_PROXY_TARGET	Frontend proxy target, typically http://backend:8000.

3.4 Common Commands

Command	Purpose
docker compose --profile app up --build	Build and start the full stack.
docker compose --profile app up	Start the stack without rebuilding.
docker compose exec backend python manage.py seed_courses	Populate the legal tutoring course catalog.
docker compose logs -f backend ai	View backend and AI-service logs.
docker compose restart ai	Restart the AI service after configuration changes.
docker compose --profile app down	Stop containers while preserving volumes.
docker compose --profile app down -v	Stop containers and delete persisted volumes.

All services run on a shared Docker network. Container environment variables should use Docker hostnames such as mongodb, backend, and ai rather than localhost. From the host browser, however, the user opens the frontend at localhost:5173. The first run is slower because the legal embedding model is downloaded and cached; later runs reuse the Docker volume.

4. Issues

Issue	Impact	Possible Solution

Issue	Impact	Possible Solution
External service dependency failures	The system depends on OpenAI, AWS S3, Qdrant Cloud, and optionally LangSmith. Missing or invalid credentials can cause degraded behavior or failed requests.	Add startup health checks, clearer UI errors, retries with backoff, and a demo mode with mocks.
RAG corpus availability	If the Qdrant collection is empty, unavailable, or misnamed, the RAG node returns no legal context and answer quality decreases.	Add a collection statistics endpoint and an ingestion verification command.
Legal reliability	LLM answers may be incomplete or jurisdictionally wrong if user facts are missing or retrieval is weak.	Add stronger disclaimers, source displays, jurisdiction confirmation, and regression tests.
PDF extraction limitations	Scanned, image-heavy, malformed, password-protected, or very large PDFs may fail or extract poorly.	Add OCR, upload validation, size limits, and document processing status.
PII anonymization edge cases	PII detection may miss uncommon identifiers or over-redact legally relevant facts.	Add legal-domain anonymization tests and configurable entity policies.
Async summarization race conditions	A user may continue chatting while background summarization is running.	Move summarization to a task queue and version summaries by message range.
Startup time	The first run downloads a large model, which may delay demonstrations.	Document expected startup time or provide a prebuilt demo image.
Testing coverage	The README does not document a full automated test strategy.	Add unit, integration, and AI evaluation tests for endpoints and pipeline nodes.
Requirement traceability	Sponsor requirement IDs are not mapped to implementation evidence in the README.	Add a requirements traceability matrix with demo steps and test evidence.

5. Outlook

If development continued, the next phase should focus on trust, evaluation, and production readiness before adding many unrelated features. The project already has a broad feature set, so the most valuable improvements are those that make the existing capabilities easier to verify, explain, and deploy safely.

Priority	Future Work	Rationale
1	Legal citation and source viewer	Show retrieved chunks, document excerpts, and source metadata beside answers to improve transparency.
2	Formal evaluation harness	Create regression tests for retrieval relevance, answer quality, anonymization, summarization, and tutoring feedback.
3	Production security hardening	Add managed secrets, HTTPS, stricter sessions, CORS hardening, API throttling, and audit logging.
4	Robust document processing	Add OCR, extraction quality warnings, page limits, and per-document status.
5	Task queue for background work	Replace background threads with Celery, RQ, or another queue for summarization and long-running jobs.
6	Admin and requirement dashboard	Show service health, Qdrant collection stats, recent LLM errors, and sponsor requirement coverage.
7	Improved tutoring personalization	Use mastery trends and FSPR deltas to recommend lessons and remedial content.
8	Deployment profiles	Add demo, development, test, and production-like Docker profiles.
9	Export features	Allow users to export chat summaries, contract reviews,

Priority	Future Work	Rationale
		and learning progress reports.
10	Human expert escalation	Package facts, documents, and questions for review by a licensed attorney when risk is high.

6. Video Presentation

YouTube link: https://www.youtube.com/watch?v=7MvoXKHh45g.